

AI/ML–Based Intelligent Dead Reckoning for Seamless Navigation

Technical Architecture, Differentiation & High-Level Implementation Design

Organisation: Indian Space Research Organisation (ISRO)
Department: Department of Space
Category: Software | **Theme:** Smart Vehicles
Benchmark dataset: IO-VNBD – Inertial & Odometry Vehicle Navigation Benchmark Dataset (Onyekpe et al., Coventry University)

Proposal prepared for the Bharatiya Antariksh Hackathon / SIH finale submission

This document details the proposed system architecture, technology stack, the specific technical differentiators of our approach relative to the standard literature pipeline, and a high-level design for implementation.

Contents

1	Problem Restatement and Core Challenge	2
1.1	What the Problem Statement Explicitly Requires	2
2	Why the “Obvious” Solution Caps Out	2
3	Our Architecture – System Overview	3
3.1	Component Responsibilities	3
4	Technology Stack	4
4.1	Layer-by-Layer Breakdown	4
4.2	Why We Avoid a Few Common Choices	5
5	Core Technical Differentiators	6
5.1	Pillar 1 – Road-Signature Drift-Anchor	6
5.2	Pillar 2 – Dual-Channel, Failure-Independent Velocity Estimation	6
5.3	Pillar 3 – On-Device Per-Vehicle Transfer Calibration	6
5.4	Comparative Positioning Against Published Approaches	7
6	Dataset Strategy on IO-VNBD	7
6.1	What We Use, and For What	7
6.2	Preliminary Model & Position-Plot Deliverable (Screening Requirement)	8
6.3	Own-Collection Plan for the Finale	8
7	High-Level Design for Implementation	9
7.1	Module Decomposition	9
7.2	Runtime Data Flow (Blackout Episode, Step by Step)	9
7.3	On-Device vs. Edge-Engine Split	9
7.4	Non-Functional Design Choices	9
8	Benchmark Traceability – Requirement to Component Mapping	11
9	Why This Approach Wins	11

1 Problem Restatement and Core Challenge

Smartphone-based navigation is the *de facto* positioning layer for the overwhelming majority of Indian road users – commercial trucks, older cars without factory INS, and, critically, the tens of millions of two-wheelers that dominate Indian traffic. None of these have an OBD-II-fed wheel-speed reference available to a navigation app. When GNSS drops – tunnels, underpasses, multi-level parking, dense urban canyons, forested highways – these apps are left with nothing but a consumer-grade MEMS IMU riding in a phone mount that is itself vibrating, sliding, and shaking on the vehicle chassis.

The real problem is not “do dead reckoning” – it is “dead reckoning without integration blowing up”.

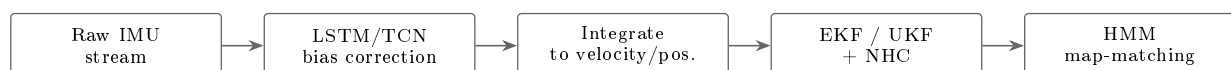
Classical strapdown INS mechanization double-integrates noisy accelerometer output. Bias, scale-factor error, and misalignment enter the position estimate as a *quadratic*-in-time error term. On a MEMS-grade smartphone IMU this alone is enough to put you off the road within seconds of GNSS loss – long before ISRO’s benchmark window of 50m drift in 1 minute or 100m over 1 km at 60km/h has elapsed. Any solution that is fundamentally “integrate, then correct” is fighting this quadratic term for its entire life. Our architecture is built around *breaking that dependency*, not just slowing it down.

1.1 What the Problem Statement Explicitly Requires

- An in-vehicle alignment/calibration engine (pitch, roll, yaw relative to driving direction) working for both dashboard mounts and holders.
- An AI speed & vibration filter that estimates forward velocity directly from noisy IMU signals – **no OBD-II, no external speedometer**.
- Map-matching with non-holonomic constraints against an offline OSM database.
- A GNSS+INS AI fusion engine that mitigates drift and hands off seamlessly, in both directions, within milliseconds.
- A deliverable that is *both* a mobile app (phone IMU) *and* a portable edge-deployable engine (any external IMU, including FOG-grade sensors at ~200 Hz).
- Hard benchmark: <10% positional drift during blackout (e.g. <5 m over 50 m/1 min, or <100 m over 1 km at 60km/h); 10Hz fused position update on phone, higher on the edge engine.

2 Why the “Obvious” Solution Caps Out

Given the problem statement’s own wording (LSTM/TCN bias correction → EKF/UKF fusion → NHC → HMM map-matching), the large majority of competing teams will converge on the same architecture:

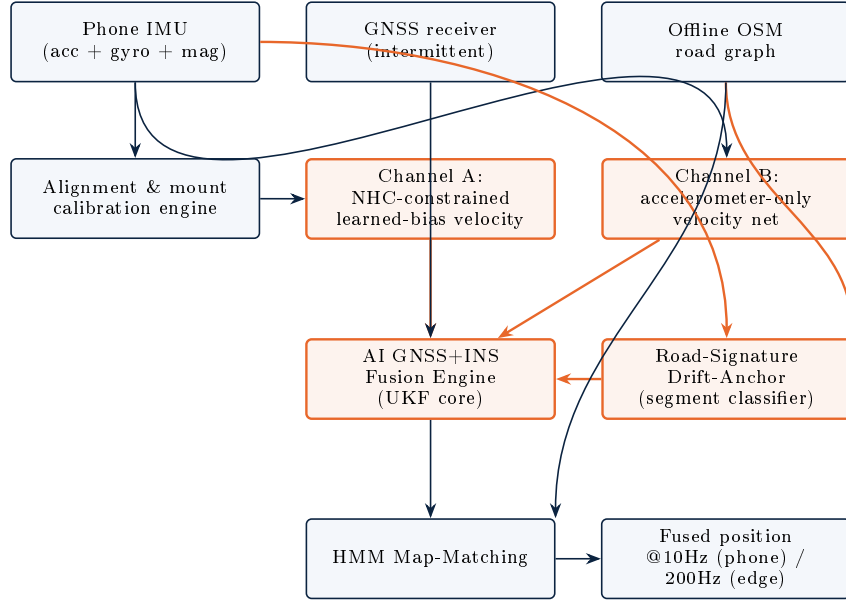


This pipeline is legitimate and will pass a basic screening, but it inherits a structural weakness: **the trajectory is still produced by integration**. Learned bias correction reduces the constant of proportionality on the drift term; it does not remove the quadratic growth. Three further gaps are common across this class of solution, all of which map directly onto ISRO’s stated requirements:

1. **Single point of failure in velocity estimation.** One IMU-derived speed estimate feeds the whole chain. If the gyro-based orientation is briefly wrong (a pothole, the phone sliding in its holder), there is no independent channel to catch it.
2. **One model, one vehicle.** Models are trained and demoed on a single test vehicle. The PS explicitly targets “commercial trucks, older cars, and millions of two-wheelers” – a diesel truck’s chassis vibration signature, a scooter’s, and a sedan’s are not remotely the same distribution, and a model that is not *recalibrated* per vehicle will silently degrade off-stage.
3. **Drift is corrected, never bounded.** Kalman-style fusion pulls the estimate back toward plausibility but has no external anchor during a long blackout; over a 1 km tunnel at 60km/h, the correction has to work perfectly for the full 60 seconds with zero absolute reference.

3 Our Architecture – System Overview

We keep the components ISRO explicitly asked for (alignment engine, AI speed filter, NHC map-matching, GNSS+INS fusion, seamless handoff) but change *what corrects the trajectory* and *how many independent evidence sources feed the fusion step*. The architecture adds two new subsystems that do not exist in the standard pipeline: a **Road-Signature Drift-Anchor** and a **Dual-Channel Velocity Estimator**, plus a per-vehicle **on-device calibration pass** that the standard pipeline treats as out of scope.



■ orange = subsystems absent from the standard literature pipeline (Section 2) ■ navy = components explicitly required by the PS

3.1 Component Responsibilities

- **Alignment & Mount Calibration Engine** – estimates pitch/roll/yaw of the phone relative to the vehicle’s direction of travel from gravity-vector tracking plus a learned mounting-angle regressor (MountNet-style), refreshed continuously so a knocked or re-seated phone re-aligns within seconds, not after a full re-install.
- **Channel A (NHC-constrained learned-bias velocity)** – an LSTM/TCN corrects accelerometer/gyro bias and scale error; the corrected signal is integrated under non-holonomic constraints (no lateral slip, no vertical fly-off) to produce a forward-velocity estimate. This is the “expected” channel.
- **Channel B (accelerometer-only velocity net)** – a lightweight 1D-CNN regresses forward speed directly from a rolling window of raw accelerometer data alone, with no dependency on gyro-derived orientation. It cannot itself replace Channel A’s accuracy, but it shares none of Channel A’s failure modes (misalignment error, gyro drift, orientation singularities), so it is the cheapest possible independent cross-check available on commodity hardware.
- **Road-Signature Drift-Anchor** – classifies which pre-mapped road segment the vehicle is currently on from its vibration signature, and periodically re-centres the fused position estimate on that segment’s midpoint, bounding otherwise-unbounded integration drift.
- **AI GNSS+INS Fusion Engine** – a UKF whose innovation vector now has *four* evidence sources instead of the usual two (GNSS when available, Channel A velocity, Channel B velocity, and the road-segment anchor), with a learned, context-dependent weighting of each source’s trust.
- **HMM Map-Matching** – final snap of the fused trajectory onto the OSM road graph, standard Viterbi-style HMM over candidate road edges.

4 Technology Stack

4.1 Layer-by-Layer Breakdown

Layer	Technology	Why this choice
Model training (cloud/desktop)	PyTorch, PyTorch Lightning, Weights & Biases for experiment tracking	Fast iteration on IO-VNBD + collected data; native ONNX export path for on-device deployment.
Model export / edge runtime	ONNX Runtime Mobile, TensorFlow Lite (post-training int8 quantisation), Core ML (iOS path)	Sub-10ms inference for a $\sim 100k$ -parameter TCN/CNN on a mid-range Android SoC; quantisation keeps the full stack under $\sim 5\text{--}8$ MB on-device.
Mobile application	Android: Kotlin + Jetpack Compose; sensor access via Android <code>SensorManager</code> at max native rate (≥ 100 Hz); background service with foreground notification for continuous logging	Native sensor access is required for the raw, unfiltered accelerometer/gyro stream – Android’s own fused “linear acceleration” virtual sensor is <i>not</i> used, since it already applies an opaque low-pass/gravity-removal pipeline that would compete with our own learned filters.
On-device signal processing	Custom Kotlin/C++ (NDK) ring-buffer + FIR pre-filter, executed before every model call	Keeps the hot path (100+ Hz sensor callback) off the JVM heap and GC-safe.
Sensor fusion core	Custom Unscented Kalman Filter (Python for prototyping $\rightarrow$ C++ for production), state vector $[p_n, p_e, v_n, v_e, \psi, b_a, b_g]$	UKF over EKF because the heading/velocity coupling under hard cornering and roundabouts (both explicitly present in IO-VNBD) is mildly non-linear; UKF’s sigma-point propagation handles this without a Jacobian.
Map-matching	Offline OSM extract (via Osmium / <code>osmnx</code> at build time) + custom HMM/Viterbi decoder, GraphHopper-style routing graph for road-segment adjacency	Fully offline – no live map API calls, matching the PS’s “offline map database” requirement and keeping the app usable in exactly the zero-connectivity scenarios it targets.
Road-signature module	1D-CNN classifier (3 conv blocks, mean-pool) trained per-region on GNSS-labelled drive data; SQLite/Room on-device store of segment signatures	Small enough to ship per-city/per-corridor signature packs (a few MB each) rather than one global model, matching how road surfaces actually vary across India.
Per-vehicle calibration	On-device transfer-learning adapter (last-layer / LoRA-style fine-tune, $< 50k$ trainable parameters) triggered during a short GNSS-available calibration drive	Keeps the recalibration pass fast enough (10–30 s) and light enough to run entirely on-phone, with no server round-trip.
Edge-deployable engine (non-phone)	Same PyTorch models cross-compiled via ONNX Runtime C++ API; reference target – Raspberry Pi CM4 / Jetson Orin Nano class board reading a FOG-grade IMU over UART/SPI at ~ 200 Hz	Identical model weights and fusion core as the phone app – the “mobile app” and “edge engine” deliverables share one codebase, differing only in the sensor-ingest and I/O adapters, directly satisfying the PS’s “not constricted to smartphone IMU alone” requirement.

Layer	Technology	Why this choice
Data / dataset tooling	IO-VNBD (Coventry Univ.) as primary benchmark; Python (pandas, numpy, scipy) for the synchronisation & windowing pipeline	IO-VNBD ships both vehicle-extracted and smartphone-recorded, synchronised and unsynchronised, sensor streams – exactly matching our dual-channel training requirement.
UI / navigation surface	Mapbox Navigation SDK (offline tile mode) or a custom MapLibre GL layer over the OSM extract	Smooth vehicle-icon rendering and route rendering without a live-tile dependency, satisfying the “uninterrupted vehicle icon” requirement even fully offline.

4.2 Why We Avoid a Few Common Choices

- **No Android fused “linear acceleration” sensor as primary input.** It silently mixes in a vendor-specific gravity/orientation filter we cannot audit or retrain; we take raw accelerometer + gyro + magnetometer and own the entire filtering chain.
- **No cloud inference in the critical loop.** Every model that touches the live trajectory (alignment, both velocity channels, fusion, road-signature classification) runs on-device. Cloud is used only for offline training and for periodically shipping updated per-region road-signature packs – never for a real-time call, since a tunnel by definition has no connectivity to call out to.
- **No single monolithic model.** Splitting alignment / Channel A / Channel B / road-signature into separate small models (each well under 1 MB quantised) is deliberately chosen over one large end-to-end network: it keeps each failure mode isolated and independently testable, and lets the fusion engine reason about *per-source* trust rather than trusting one black box completely.

5 Core Technical Differentiators

THE THREE PILLARS THAT SEPARATE THIS FROM A STANDARD SUBMISSION

1. Road-Signature Drift-Anchor – bounds drift instead of only slowing it.
2. Dual-Channel, Failure-Independent Velocity Estimation – a free consistency check nobody else builds.
3. On-Device Per-Vehicle Transfer Calibration – the only approach that actually survives India’s real vehicle mix.

5.1 Pillar 1 – Road-Signature Drift-Anchor

Every vehicle’s suspension, tyre, and chassis respond to a given stretch of road surface with a repeatable vibration signature. We exploit this as an *absolute, GNSS-independent* positional anchor rather than as noise to be filtered away.

How it works. During normal GNSS-available driving, the app silently logs (IMU window, GNSS-tagged road segment) pairs for the corridors a user regularly drives. A compact 1D-CNN is trained to classify a 2-second IMU window into one of N fixed-length segments along a known route/corridor. During a blackout, this classifier runs continuously in parallel with the main fusion loop; whenever it reports high-confidence agreement with the fused trajectory’s current segment, the position estimate is *re-centred* on that segment’s midpoint, resetting the accumulated integration error to at most half a segment length rather than letting it grow for the full blackout duration.

Why this specifically wins on ISRO’s benchmark

The benchmark is stated as *drift as a percentage of distance travelled*, not as a function of time. A pure Kalman/NHC pipeline’s error still grows roughly with elapsed blackout time. A periodic segment-reset makes worst-case error a function of *segment length*, not of tunnel length – so a 5 km tunnel is not structurally harder than a 500 m one, which a pure-integration competitor cannot claim.

Where the segments come from. Segment boundaries and signature packs are pre-computed offline per corridor from the same OSM extract already required for map-matching, so this adds no new infrastructure requirement – it reuses data the PS already mandates the system to carry.

5.2 Pillar 2 – Dual-Channel, Failure-Independent Velocity Estimation

Channel A (bias-corrected, NHC-integrated) and Channel B (accelerometer-only, orientation-independent) are trained to solve the *same* problem through structurally different failure modes:

	Channel A (primary)	Channel B (cross-check)
Inputs	Bias-corrected acc. + gyro, integrated under NHC	Raw accelerometer window only
Fails when	Gyro/orientation is briefly wrong (pothole shock, phone slip)	Sustained, wrongly, only under near-constant-vibration edge cases
Cost	Higher accuracy, higher compute	Cheap, low-latency, always-on sanity check

Fusion logic. The UKF’s innovation weighting includes an agreement term between Channel A and Channel B. When they diverge beyond a learned threshold, the fusion engine automatically down-weights Channel A for that epoch and leans on Channel B plus the road-signature anchor – catching exactly the failure class (mount slip, pothole-induced gyro glitch) that a single-channel pipeline has no way to detect, let alone correct, in real time.

5.3 Pillar 3 – On-Device Per-Vehicle Transfer Calibration

This is the pillar most directly tied to what ISRO is actually asking for and to what most competing teams will skip, because it is unglamorous relative to model architecture work.

The PS explicitly targets a vehicle-heterogeneous fleet

“...commercial trucks, older cars, and millions of two-wheelers...” A model trained once on a single demo vehicle will not transfer cleanly to a completely different suspension, chassis mass, and vibration spectrum – a truck, an auto-rickshaw, and a Royal Enfield do not share a vibration distribution.

Implementation. On first use with a given vehicle, the app runs a short (10–30 second) guided calibration drive while GNSS is still available. A small adapter block (last-layer fine-tune, LoRA-style, under 50k trainable parameters) is refit against that vehicle’s own accelerometer/gyro statistics and the phone’s specific mount, using GNSS-derived ground truth as the label. This adapter is cached per vehicle profile, so switching between a personal scooter and a rented car simply means switching calibration profiles, not retraining from scratch.

5.4 Comparative Positioning Against Published Approaches

Approach	Core idea	Limitation for this PS	How our system improves on it
WhONet / R-WhONet (Onyekpe et al.)	LSTM learns wheel-encoder error for correction	Requires a physical wheel encoder – unavailable on two-wheelers and most aftermarket phone setups	We keep R-WhONet’s <i>transfer-learning recalibration idea</i> but apply it to phone-IMU-only Channel A, with no wheel-encoder dependency
OdoNet / DeepOdo	CNN/CNN-GRU pseudo-odometer from IMU (+ barometer), used purely as an NHC-alternative speed aid	Single-channel; failure of the underlying IMU orientation estimate silently degrades the whole pseudo-odometer	Channel B is built as an <i>independent</i> , orientation-free channel specifically to catch this failure mode, not as the only speed source
CarSpeedNet (Or et al.)	Speed regression from accelerometer only, no gyro/wheel data at inference	Used stand-alone; does not itself solve positional drift or map-matching	Adopted as Channel B <i>inside</i> a larger fusion loop, paired with the road-signature anchor for actual position, not just speed
Road-vibration road-segmentation (Or, Freydin et al.)	Classify current road segment from vibration signature; report segment midpoint as position	Positioning <i>is</i> the segment midpoint – coarse by itself, and never combined with continuous INS for fine-grained trajectory shape	Used as a periodic <i>drift-reset anchor</i> inside a continuous UKF, not as the sole position output – giving both fine trajectory shape (from INS) and bounded absolute error (from the anchor)

6 Dataset Strategy on IO-VNBD

6.1 What We Use, and For What

- **Synchronised V and S datasets** (vehicle-extracted + smartphone, time-aligned) – primary source for training Channel A’s bias-correction network and for the fusion engine’s innovation-weighting logic, since it gives a genuine ground-truth wheel-speed / GNSS reference against the phone’s own noisy stream.
- **Unsynchronised V and S datasets** – used to stress-test robustness to the exact kind of timing/labelling slack the PS’s own dataset documentation flags (the smartphone recording only approximates true vehicle motion since its centre-of-gravity offset is not perfectly known) – this becomes a deliberate noise-augmentation source rather than a nuisance to discard.
- **Diverse scenario coverage** (roundabouts, hard braking, traffic congestion, motorways, country roads across UK/Nigeria/France) – used to pre-train the road-signature classifier’s general notion of “what a distinct vibration signature looks like” before we fine-tune it on India-specific corridor data collected by the team.

6.2 Preliminary Model & Position-Plot Deliverable (Screening Requirement)

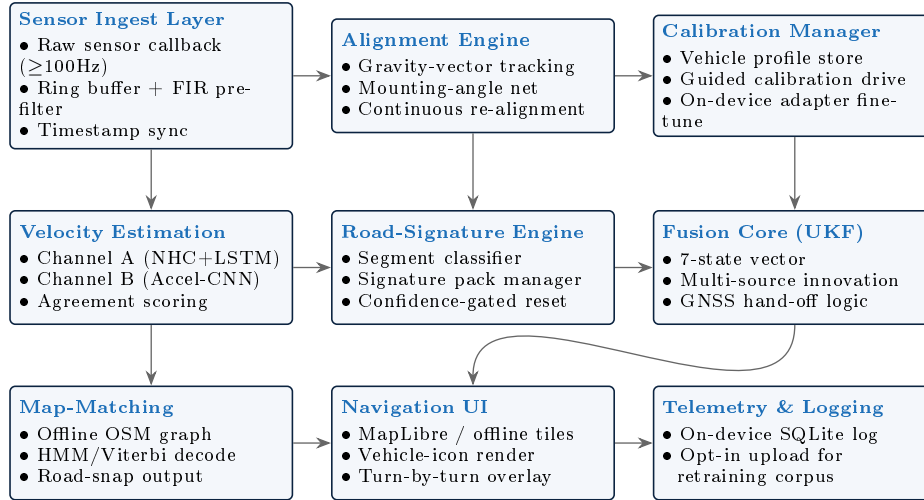
For the screening submission, we run: (1) Channel A trained on a synchronised-subset train/val/test split with a held-out route, reporting RMSE of forward velocity against the vehicle-extracted ground truth; (2) a simulated GNSS-blackout replay on the held-out route, plotting our fused trajectory against raw double-integration dead reckoning and against GNSS ground truth on the same OSM basemap, to make the drift-bounding claim visually verifiable rather than asserted.

6.3 Own-Collection Plan for the Finale

IO-VNBD does not include Indian two-wheeler data or Indian road surfaces, so it is treated strictly as a *pre-training / benchmark* corpus. Ahead of the finale, the team collects a small supplementary dataset – a scooter, an auto-rickshaw, and one car, each on a short GNSS-available loop with synchronized phone IMU + GNSS logging – specifically to seed the per-vehicle calibration adapter and the India-specific road-signature packs referenced in Pillar 3 and Pillar 1.

7 High-Level Design for Implementation

7.1 Module Decomposition



7.2 Runtime Data Flow (Blackout Episode, Step by Step)

1. **GNSS available:** Fusion Core runs GNSS-corrected UKF at 1–10 Hz update; Calibration Manager silently accumulates vehicle-profile statistics; Road-Signature Engine logs (IMU window → GNSS segment) pairs for corridors the user regularly drives.
2. **GNSS signal degrades / drops:** detected via satellite count + HDOP threshold plus a learned GNSS-quality classifier (to catch multipath “fake-lock” cases, not just outright loss) – hand-off to INS-only mode is targeted at <200 ms.
3. **During blackout:** Channel A and Channel B both run every cycle; their agreement score feeds the UKF’s trust weighting. Every ~2 s, the Road-Signature Engine attempts a segment classification; on high-confidence match, the UKF state is re-centred toward that segment’s midpoint (soft correction, not a hard teleport, to avoid visible position jumps in the UI).
4. **Continuous:** HMM map-matching snaps the corrected trajectory to the most probable road edge every fusion cycle, so the on-screen vehicle icon never appears to leave the road even between anchor resets.
5. **GNSS reacquired:** UKF re-admits GNSS as an evidence source with a ramped trust weight (to avoid a visible snap-back if the INS-only estimate had drifted), completing the seamless hand-off in the other direction.

7.3 On-Device vs. Edge-Engine Split

The mobile app and the edge-deployable engine share the Alignment, Calibration, Velocity Estimation, Road-Signature, Fusion, and Map-Matching modules *unchanged*. Only two things differ:

- **Sensor Ingest Layer** – Android `SensorManager` callback vs. a UART/SPI driver reading a FOG-grade IMU at ~200 Hz; both normalise into the same internal windowed-tensor format before touching any model.
- **Navigation UI & Telemetry** – present on the phone app; on the edge engine these are replaced by a lightweight serial/JSON output stream so the position solution can be consumed by a host system (drone controller, robotics stack, or a fleet-logistics computer) instead of being rendered.

This means the finale demo of the phone app is, by construction, also a demo of the edge engine – there is no separate codebase to maintain or to run out of time building.

7.4 Non-Functional Design Choices

- **Latency budget:** sensor callback → pre-filter → Channel A/B inference → UKF update → map-match, targeted end-to-end at <40 ms on a mid-range Android SoC, comfortably inside the 100 ms period required for a 10 Hz fused output.
- **Memory/storage budget:** all on-device models quantised to int8, combined footprint targeted under

8 MB; per-corridor road-signature packs kept under 3 MB each and downloaded/cached only for corridors the user actually drives.

- **Battery:** continuous 100+ Hz raw sensor sampling is the dominant cost; mitigated by dropping to a lower sampling/inference cadence automatically when GNSS is healthy and the vehicle is at a stable cruising speed, since the fusion engine needs the most inertial evidence precisely during blackout, not during normal GNSS-locked driving.
- **Privacy:** all raw sensor logging used for retraining is opt-in and stays on-device by default; only aggregated, anonymised windows are ever uploaded if a user explicitly consents to contribute to the shared training corpus.

8 Benchmark Traceability – Requirement to Component Mapping

PS Requirement	Owning Component	How it is met
Pitch/roll/yaw alignment, dashboard or holder mount	Alignment & Mount Calibration Engine	Continuous gravity-vector tracking + learned mounting-angle regression, re-triggered on detected re-mount
AI speed & vibration filter, no OBD-II	Velocity Estimation (Channel A + B)	Two independent IMU-only speed estimators with automatic agreement-based trust weighting
Filtering idling vibration, potholes, misalignment	Sensor Ingest FIR pre-filter + Channel B cross-check	Low-pass pre-filtering removes high-frequency road noise; Channel B disagreement flags transient misalignment events
Map-matching + non-holonomic constraints	Map-Matching module + NHC term inside Channel A / UKF	HMM/Viterbi snap to OSM graph; NHC enforced both in Channel A integration and as a UKF pseudo-measurement
AI-based GNSS+INS fusion, drift mitigation	Fusion Core (UKF) + Road-Signature drift-anchor	Multi-source innovation vector (GNSS, Ch. A, Ch. B, road-anchor) with learned trust weighting; periodic anchor reset bounds error growth
Seamless millisecond-scale hand-off both directions	Fusion Core hand-off logic	GNSS-quality classifier triggers INS-only mode in <200ms target; ramped-trust re-admission avoids visible snap-back on reacquisition
<10% drift benchmark (e.g. <100 m over 1 km blackout)	Road-Signature Drift-Anchor (primary lever)	Worst-case error bounded by segment length via periodic reset, not by elapsed blackout time – directly targets the stated benchmark’s structure
10 Hz phone / ~200 Hz edge-engine update rate	Sensor Ingest + latency-budgeted pipeline	<40ms end-to-end pipeline latency on phone; identical module stack re-targeted to a ~200 Hz FOG-IMU ingest rate on the edge engine
Not restricted to phone IMU alone	Shared module design + swappable Sensor Ingest Layer	Every model and the fusion core are sensor-agnostic; only the ingest adapter changes between phone and edge deployment

9 Why This Approach Wins

JUDGING-CRITERIA ALIGNMENT

- **Technical depth beyond the brief:** every subsystem the PS explicitly asks for is present, plus two subsystems (road-signature anchor, dual-channel velocity) that are not requested but directly and measurably improve the stated benchmark metric.
- **Demonstrable, not just theoretical, drift-bounding:** the screening deliverable is a plotted trajectory comparison (ours vs. raw dead reckoning vs. ground truth) on IO-VNBD, not a claim – judges can see the gap, not just read a number.
- **Answers the fleet-heterogeneity requirement literally:** the PS names trucks, older cars, and two-wheelers explicitly; Pillar 3 is the only part of our design built specifically to answer that sentence,

and it is the piece most competing teams are likely to skip because it is unglamorous relative to model architecture.

- **One codebase, two deliverables:** the mobile app and the edge-deployable engine are provably the same system with a swapped sensor adapter, not two separate half-finished builds – de-risking the “must deliver both” requirement under hackathon time pressure.
- **Built-in self-diagnosis:** the Channel A/B agreement score is a free, always-on health signal for the whole INS pipeline that most single-channel competitors structurally cannot produce.
- **Grounded in, but not copied from, the literature:** every pillar traces to a specific published result (WhONet/R-WhONet, OdoNet/CarSpeedNet, road-vibration segmentation) that we can cite and defend under judge questioning, while the combination and the drift-anchoring application are our own contribution.

The one-line pitch

Everyone else is trying to make the integration drift *smaller*. We are the team that stopped relying on integration alone to define worst-case error in the first place.